

My Module: Student Dashboard (End-to-End) — 2-Day Build Plan

Scope: Full stack — backend APIs and frontend UI for the Student (Applicant) role only.

0. Requirements & Specifications

Functional Requirements

ID	Requirement
FR-1	Student can register/log in and see only their own data
FR-2	Student can browse a list of active scholarships (title, amount, eligibility summary, deadline)
FR-3	Student can view full detail of a scholarship before applying
FR-4	Student can fill an application form (academic details, income, category) scoped to a specific scheme
FR-5	Student can upload required documents (marksheet, income certificate, ID proof, etc.)
FR-6	Student can submit the application, after which it becomes read-only (locked)
FR-7	Student can view a status tracker showing current stage: Submitted → Under Review → Verified → (Pending Admin Approval →) Approved/Rejected → Disbursed
FR-8	If a document is flagged invalid, student sees which one and why, and can re-upload only that document
FR-9	Student can view a notifications inbox (read/unread) for every status change
FR-10	Student can view history of all past applications across schemes

Non-Functional Requirements

ID	Requirement
NFR-1	Student can only ever access their own applications — enforce via <code>studentId</code> match server-side, not just frontend filtering
NFR-2	File upload restricted by type (PDF/JPG/PNG) and size (e.g., max 5MB), validated both client and server side
NFR-3	Form validation prevents submission with missing required fields/documents
NFR-4	Status tracker updates without requiring a page refresh (poll or simple re-fetch on mount is fine for hackathon scope)

What You're Explicitly NOT Building

- Scholarship scheme creation (Admin module)
- Document verification logic/UI (Staff module)
- Approval/rejection/disbursement actions (Admin/Staff modules)
- Notification delivery mechanism — you just display what's already been created for you

1. Module Breakdown

- **Module A** — Scholarship Browse & Detail View
- **Module B** — Application Form + Document Upload
- **Module C** — Status Tracker
- **Module D** — Resubmission Flow
- **Module E** — Notifications Inbox + Application History

2. Day-by-Day Task List

DAY 1 — Morning (Setup + Module A)

- ☐ Sync with Auth owner: confirm JWT payload and reusable role middleware
- ☐ Confirm `Scholarship` schema field names with Admin module owner (you're a pure consumer of this table — get exact field names now, not later)
- ☐ Build API: `GET /api/scholarships?status=active` (public list for students)
- ☐ Build API: `GET /api/scholarships/:id` (detail view)
- ☐ Frontend: Scholarship list page (cards: title, amount, deadline, quick eligibility summary)
- ☐ Frontend: Scholarship detail page with "Apply" button

DAY 1 — Afternoon (Module B)

- ☐ Build API: `POST /api/applications` (create application, body = form data + scholarshipId)
- ☐ Build API: `POST /api/applications/:id/documents` (file upload, multipart — use Multer)
- ☐ Frontend: Multi-field application form (academic details, income, category) with client-side validation
- ☐ Frontend: Document upload widget (drag-drop or file picker, shows required doc types per scheme)
- ☐ Frontend: Submit confirmation → lock form after submission

DAY 1 — End of day checkpoint

- ☐ Demo scholarship browse → apply → upload flow to team lead. Confirm document upload response shape matches what Staff module expects to render (this is the #1 handoff point — nail it early)

DAY 2 — Morning (Module C)

- ☐ Build API: `GET /api/applications/my` (student's own applications, paginated)
- ☐ Frontend: Status tracker component — visual stepper (Submitted → Under Review → Verified → Pending Admin Approval → Approved → Disbursed), highlighting current stage
- ☐ Frontend: Show staff/admin remarks if present (read-only)

DAY 2 — Midday (Module D — Resubmission)

- ☐ Build API: `GET /api/applications/:id` (detail, to show which document(s) were flagged invalid + reason)
- ☐ Build API: `PATCH /api/applications/:id/documents/:docId/resubmit` (re-upload single document)
- ☐ Frontend: If any document status = `invalid`, show inline "Reupload" action with the staff's remark visible
- ☐ Verify this correctly flips application status back to `under_review` (test with Staff module owner)

DAY 2 — Afternoon (Module E + Polish)

- ☐ Build API: `GET /api/notifications/my`, `PATCH /api/notifications/:id/read`
- ☐ Frontend: Notification bell + dropdown/inbox page, unread count badge
- ☐ Frontend: Application history list (all past applications, filter by status)
- ☐ Full integration test: apply → see it appear correctly in Staff's queue → get flagged → resubmit → get re-verified → see status update on your tracker
- ☐ UI polish: loading/empty states, mobile-responsive form and upload widget

DAY 2 — Final hours

- ☐ Deploy/confirm merge
- ☐ Prepare demo segment: apply for a scholarship live, then switch to Staff/Admin view (teammate's screen) to show the same application appearing there — this cross-role visibility is your strongest demo moment
- ☐ Rehearse: *"How do you prevent a student from seeing another student's application?"* (answer: server-side `studentId` check on every route, not just UI hiding)

3. API Contract You Need From Teammates (Day 1 AM handshake)

From...	What
Auth owner	JWT structure, role middleware
Admin module owner	Exact <code>Scholarship</code> field names (<code>eligibilityCriteria</code> , <code>amount</code> , <code>deadline</code> , <code>requiresAdminApproval</code>) so your form/display doesn't mismatch
Staff module owner	Exact status strings used when they flag a document invalid, and the remark field name, so your resubmission UI reads them correctly

4. Your API Endpoints — Full List

```
GET    /api/scholarships?status=active
GET    /api/scholarships/:id
POST   /api/applications
POST   /api/applications/:id/documents
GET    /api/applications/my
GET    /api/applications/:id
PATCH /api/applications/:id/documents/:docId/resubmit
GET    /api/notifications/my
PATCH /api/notifications/:id/read
```